

CS3101: Operating System Lab

05th August, 2026

Lab 2: Booting Unraveled

Study Material

General Instructions

- This companion document follows the tasks and terminology used in the laboratory assignment.
- It explains the required concepts, design decisions, testing strategy, and debugging approach.
- It intentionally does not provide a complete bootloader, complete kernel, complete linker script, or ready-to-submit solution.
- Any code fragments are incomplete templates. Replace every TODO after understanding the purpose of the missing step.
- Write your own report, preserve proper indentation, and explain why each low-level operation is required.

Booting Unraveled - Learning Objective

The assignment traces the complete path from the BIOS loading a 512-byte boot sector to a small 32-bit protected-mode kernel that communicates directly with VGA and keyboard hardware. The central learning goal is to understand what normally happens before a conventional operating system begins: binary layout, disk-sector loading, memory placement, validation, protected-mode entry, freestanding C execution, and direct hardware access.

Recommended Working Method

1. Complete and test one checkpoint before beginning the next checkpoint.
2. Keep separate binaries for Stage 1, Stage 2, and the kernel, and verify the exact byte size of each binary.
3. Use hexadecimal dumps and disassembly whenever an address, offset, jump target, or magic value is uncertain.
4. Add short progress messages at important transitions, but remove unnecessary output after debugging.
5. Maintain a memory map and disk layout on paper while implementing the loader.

System Roadmap

```
BIOS startup
-> load LBA 0 at physical address 0x7C00
Stage 1
-> read LBA 1-4 into physical address 0x8000
-> validate the STG2 header
Stage 2
-> read LBA 5-36 into physical address 0x10000
-> validate the KRNL header
-> enable A20, install GDT, enter protected mode
Kernel entry at 0x10004
-> initialize C environment
-> VGA terminal + PS/2 keyboard + command shell
```

Important Principle

At every boundary, validate before transferring control. A successful disk interrupt means that the BIOS reported success; it does not independently prove that the intended component was loaded at

the intended location. The four-byte STG2 and KRNL headers provide a simple identity check before a jump or call is made.

Foundation Concepts Required for All Tasks

1. Binary sectors and exact sizes

A disk sector is treated as 512 bytes in this lab. The Stage 1 boot sector therefore occupies exactly one sector. Stage 2 occupies four sectors, and the kernel occupies 32 sectors. Padding is not decorative: it ensures that the later component begins at the LBA assumed by the loader.

Component	Disk placement	Required size	Load address
Stage 1	LBA 0	1 sector = 512 bytes	0x7C00
Stage 2	LBA 1-4	4 sectors = 2048 bytes	0x8000
Kernel	LBA 5-36	32 sectors = 16384 bytes	0x10000

Check yourself: If Stage 2 accidentally becomes five sectors, the kernel no longer begins at LBA 5. The Stage 2 read may still succeed, but the kernel loader will read the wrong bytes.

2. File offsets, runtime addresses, and LBAs

These are three different coordinate systems. A file offset counts bytes from the start of one binary. A runtime address identifies a memory location after loading. An LBA identifies a sector on the disk image. Do not use them interchangeably.

Quantity	Example	Meaning
File offset	0x0010	The seventeenth byte inside a binary file
Runtime address	0x7C10	A byte in memory after the boot sector is loaded
LBA	5	The sixth sector of the raw disk image

For a boot sector assembled with origin 0x7C00, a byte at file offset 0x0010 is normally located at runtime address 0x7C10. However, disassembly tables requested by the assignment must report file offsets starting at 0x0000.

3. Little-endian storage

x86 stores multi-byte integer values with the least significant byte first. Consequently, a 16-bit word written in assembly may appear reversed in a hexadecimal dump. This explains why the boot-signature word conventionally written as 0xAA55 appears as the final byte sequence 55 AA.

4. Segment:offset addressing in real mode

In real mode, a physical address is obtained from a segment and offset. The conceptual calculation is:

```
physical address = (segment << 4) + offset
```

For the simple memory map used here, setting the relevant segment registers to zero makes offsets such as 0x7C00, 0x8000, and 0x1000 directly represent low physical addresses. Loading 0x10000 requires a non-zero segment:offset pair because a 16-bit offset alone cannot contain 0x10000.

Design hint: Represent the kernel destination 0x10000 as a segment:offset pair that evaluates to the same physical address. Verify the arithmetic before placing it in the Disk Address Packet.

5. BIOS services versus direct hardware access

Execution environment	Available output method	Disk/keyboard method
16-bit real mode	BIOS INT 0x10	BIOS INT 0x13 and BIOS facilities
32-bit protected mode	Write to 0xB8000	Read PS/2 I/O ports directly

The assignment deliberately crosses this boundary. Once protected mode is entered, continue with the kernel's own VGA and keyboard routines rather than calling real-mode BIOS interrupts.

6. Safe halt loops

A single HLT instruction does not necessarily halt forever: an enabled interrupt can wake the processor. A permanent stop path therefore disables maskable interrupts, halts, and loops back to the halt location. Use the same concept for fatal validation failures and for the kernel halt command.

Task 1: Boot Forensics

Goal of the study material: Understand how source instructions become raw bytes, how a BIOS recognizes a bootable sector, and how to inspect and modify a compiled image without confusing file offsets with runtime addresses.

Task 1.1 - Designing the two boot sectors

Begin with the smallest possible real-mode program structure. Both source files should contain the same initialization and message-printing logic. The deliberate difference should be confined to the final two signature bytes.

Suggested internal structure

```
[16-bit mode and origin directives]
[entry label]
    disable interrupts while preparing stack
    initialize DS, ES, SS and SP
    clear direction flag
    re-enable interrupts
    point SI to a null-terminated message
[character loop]
    TODO: fetch one byte, test for zero, print, repeat
[safe halt loop]
[message bytes + terminating zero]
[padding expression]
[two-byte signature]
```

This is a structural outline only. You must decide the actual instructions, labels, jump directions, and register-preservation choices.

Processor-state initialization hints

- Use a known value for DS and ES before dereferencing message labels or using string instructions.
- Change SS and SP while maskable interrupts are disabled. An interrupt between changing SS and changing SP can make the processor use a mismatched stack segment and stack pointer.
- Set SP to a location that does not overwrite the executing boot-sector code during ordinary pushes and calls.
- Clear the direction flag before using LODSB so that SI advances toward the terminating zero byte.

Message-printing loop hints

The BIOS teletype service prints one character at a time. Keep the string address in SI, load one byte into AL, terminate when AL is zero, select the teletype function in AH, and invoke the BIOS video interrupt. The loop should not print the terminating zero.

Padding and signature reasoning

The binary must reserve all bytes up to the final two positions and then place the signature there. Rather than manually counting zeros, use an assembly-time expression based on the current output position. Confirm the result with a file-size command and a dump of the last bytes.

Do not copy blindly: Derive why the padding expression subtracts the current assembled size and also reserves two bytes for the final signature. A wrong expression may create a 510-byte or 514-byte image.

Task 1.2 - Assembly and size verification

Use NASM flat-binary output, not ELF object output. A flat binary contains exactly the bytes intended for the disk sector and has no executable-file headers. Verify the result in two independent ways: inspect the reported file size and dump the final bytes.

```
nasm -f bin <source.asm> -o <output.bin>
stat -c %s <output.bin>
xxd -g 1 <output.bin> | tail
```

The placeholders above must be replaced with your filenames. The size should be interpreted in decimal bytes, while xxd offsets are normally shown in hexadecimal.

Task 1.3 - Mapping regions in the binary

Identify the four required regions by combining the assembler listing, a hexadecimal dump, the known source order, and the null-terminated message. Do not guess the code boundary from the first zero byte because a machine instruction may legitimately contain a zero operand byte.

Region	How to locate it	What to record
Machine code	Use a listing or disassembly; begin at file offset 0	First and last instruction-byte offsets
Message	Search for the ASCII byte sequence and its final 00	Message start through terminator
Padding	Repeated zero bytes after meaningful data	First padding byte through offset 0x01FD
Signature	Final two bytes	Offsets 0x01FE and 0x01FF

ASCII-location hint

Each printable character corresponds to an ASCII byte. You may use a short text-to-hex reference or the right-hand ASCII column of xxd. The terminator immediately after the last character must be included in the message region.

Disassembly hint

Specify 16-bit disassembly and, when desired, an origin matching the BIOS load address. Remember that the assignment table asks for file offsets beginning at zero. If the disassembler prints runtime addresses beginning near 0x7C00, subtract the origin or use a mode that reports file-relative offsets.

```
ndisasm -b 16 -o 0x7C00 <binary>
# Use the listing or arithmetic to convert displayed addresses to file offsets.
```

Choosing five instructions for the table

Choose instructions of different types so the comparison is meaningful: a one-byte control instruction, a register-to-register instruction, an immediate move, a conditional jump, and an interrupt instruction. For every entry, verify the exact byte count; otherwise all subsequent offsets in the table will be shifted.

Task 1.4 - Predicting BIOS behaviour

Write the prediction before running QEMU. The valid image should be considered bootable because the BIOS finds the expected two-byte marker at the end of the 512-byte sector. The corrupt image may cause the BIOS to continue to another boot device or print a no-bootable-device message. The exact firmware wording can vary, so distinguish the principle from the particular QEMU display.

Testing hint

```
qemu-system-i386 -drive format=raw,file=<image.bin>  
# Use a separate run for each image and capture the visible result.
```

If QEMU treats a 512-byte file as a floppy by default on your system, use an explicit raw drive option so that the intended boot-drive behaviour is clear and repeatable.

Task 1.5 - Patching one compiled byte

First locate the exact file offset of a visible character in the stored message. Copy the valid binary, then overwrite exactly one byte in the copy. Use a command configuration that does not truncate the file and writes one byte at the chosen offset.

```
cp valid_boot.bin patched_boot.bin  
printf <one-byte-value> | dd of=patched_boot.bin bs=1 seek=<offset> conv=notrunc  
strings patched_boot.bin
```

The snippet intentionally omits the replacement byte and offset. Determine both from your dump. After patching, verify that the file remains 512 bytes and that its final signature bytes are unchanged.

Explanation hint

Changing a message character modifies ordinary payload data. The processor can still execute the surrounding instructions, and the BIOS marker remains intact. Changing one of the final signature bytes changes the metadata that the BIOS uses to decide whether to execute the sector at all.

Task 1 debugging checklist

- The source is assembled with 16-bit instruction encoding and the correct origin.
- DS points to the segment used by the assembled message address.
- The message ends with a zero byte.
- The program halts instead of falling through into text or padding.
- Each binary is exactly 512 bytes.
- The valid image ends in 55 AA; only the corrupt image differs there.
- Reported offsets are file offsets, not runtime addresses.

Task 2: Reusable Real-Mode Console

Goal of the study material: Replace repeated BIOS-output sequences with named procedures, preserve the BIOS boot-drive value, and correctly convert an 8-bit value into two uppercase hexadecimal characters.

Task 2.1 - Preserving the boot drive and initializing state

The BIOS places the boot-drive identifier in DL before transferring control. Save it immediately into a byte variable, before using a procedure or BIOS service that might overwrite DX. Later disk operations must reload DL from this saved variable rather than assuming a fixed value such as 0x80.

Why not hard-code 0x80?

0x80 commonly represents the first hard disk, but the loader should remain correct when started from a different BIOS drive. The saved value is part of the calling contract between the BIOS and the boot sector.

Safe stack initialization sequence

```
disable maskable interrupts
set AX to the desired segment value
load DS and ES
load SS
immediately load SP
clear direction flag
re-enable maskable interrupts
```

This is sequencing guidance, not complete assembly. Explain in the report that an interrupt pushes a return frame using SS:SP; therefore an interrupt during a partially updated stack can write to an unintended memory location.

Task 2.2 - Procedure contracts

Treat every procedure as an interface with defined input, output, and clobbered registers. A procedure that works only because the caller happens to leave a register unchanged is fragile. Add a short comment above each routine stating which registers it reads and modifies.

Procedure	Input contract	Core idea	Registers to consider
print_char	AL = character	Only location containing BIOS INT 0x10	AH and BIOS side effects
print_string	DS:SI = zero-terminated string	Fetch, test, call print_char, repeat	AX and SI
print_newline	No input	Print CR then LF through print_char	AL
print_hex8	AL = byte	Split into high and low nibbles	AX and temporary storage

Hint for print_char

Place the BIOS teletype function number in AH and invoke INT 0x10. Because this must be the only direct video interrupt in the program, all other output routines should call this procedure even when printing fixed control characters.

Hint for print_string

```
repeat:
    load the next byte from DS:SI
    if the byte is zero, return
    call print_char
    continue
```

Decide whether print_string promises to preserve SI. Either choice can be valid if the contract is documented and callers are written consistently. In a small boot sector, conserving bytes may be important, but correctness comes first.

Hint for print_newline

A terminal newline in this BIOS context is conventionally formed by printing carriage return 0x0D followed by line feed 0x0A. Printing only line feed may move down without returning to the first column.

Hint for print_hex8

A byte contains two four-bit nibbles. The high nibble is obtained by shifting or rotating the original byte right by four positions and masking to the range 0-15. The low nibble is obtained by masking with 0x0F. Preserve the original byte or save the low nibble before transforming the high nibble.

Nibble-to-ASCII reasoning

Nibble range	ASCII conversion idea	Examples
0-9	add the ASCII value of '0'	0 -> 0, 5 -> 5
10-15	convert relative to 'A' after accounting for 10	10 -> A, 15 -> F

Implement the conversion once in a helper routine if that makes the logic clearer. The output must always contain two characters, so a value such as 0x05 must print 05 rather than 5.

Procedure-preservation strategy

- Write down each routine's input and outputs before coding it.
- Save a register with PUSH only when the stack is already valid.
- Restore registers in reverse order before RET.
- Do not accidentally restore AL in print_char before invoking the BIOS, because AL is the input character.
- Test procedures individually with fixed values before using the saved boot-drive byte.

Task 2.3 - Structured diagnostic output

Separate static prefixes from dynamic values. For the boot-drive line, store and print a static prefix ending in 0x, then load the saved boot-drive byte into AL and call print_hex8. Do not embed the characters 80 in the message because that would not demonstrate use of the BIOS-supplied value.

Suggested call sequence

```
print first message
print newline
print student-ID prefix and ID text
print newline
print boot-drive prefix
load saved boot-drive byte into AL
call print_hex8
enter permanent safe halt loop
```

Testing print_hex8 independently

Before integrating the boot-drive output, temporarily test values that exercise both conversion branches: a leading-zero value, a decimal-only value, and a value containing A-F. Remove or disable these temporary tests in the final assignment output.

Test category	Representative value	What it verifies
Leading zero	0x05	Exactly two digits
Boundary	0x09 and 0x0A	Transition from 9 to A
Both alphabetic	0xAF	High and low A-F conversion

Task 2 debugging checklist

- The first instruction sequence saves DL before other code can modify it.
- INT 0x10 appears only inside print_char.
- Every string has a terminating zero.

- `print_newline` outputs CR before LF.
- `print_hex8` outputs uppercase and preserves any registers promised by its contract.
- The boot-drive digits are generated dynamically from the saved byte.
- The final halt path cannot fall through into data.

Task 3: Robust Multi-Stage Loader

Goal of the study material: Build a verified disk-loading chain, implement bounded retries, validate component identity with magic bytes, and perform the transition from real mode to a flat 32-bit protected-mode environment.

Memory and disk plan

Before writing code, create a fixed layout and check for overlap. Stage 1 executes near 0x7C00, Stage 2 is loaded at 0x8000, the kernel begins at 0x10000, and the protected-mode stack is placed at 0x90000. Keep the real-mode stack below the boot sector unless your design explicitly reserves another safe region.

Range or address	Purpose	Main risk
0x7C00 area	Stage 1 code/data and initial stack boundary	Stack overwriting Stage 1
0x8000-0x87FF	Four-sector Stage 2 image	Wrong DAP destination
0x10000-0x13FFF	32-sector kernel image	Wrong LBA or segment:offset
0x90000 downward	Protected-mode stack	Incorrect data selector or ESP

Task 3.1 - Stage 1 loading Stage 2

Stage 1 must fit into 512 bytes, so keep its design compact and modular. It needs processor initialization, a saved boot drive, a DAP, a bounded retry loop, minimal output routines, magic validation, and a final transfer to Stage 2.

Understanding the 16-byte Disk Address Packet

BIOS extended read service AH=0x42 expects DS:SI to point to a packed 16-byte structure. Build the structure in the exact field order and width required by the BIOS interface. Multi-byte fields are little-endian.

Field	Size	Stage 1 value concept
Packet size	1 byte	Size of the DAP structure
Reserved	1 byte	Zero
Sector count	2 bytes	Four sectors
Destination offset	2 bytes	Offset part of 0x8000
Destination segment	2 bytes	Segment part of 0x8000
Starting LBA	8 bytes	LBA 1

DAP design checks

- Ensure DS:SI actually points to the DAP at the moment INT 0x13 is invoked.
- Reload DL from the saved boot-drive variable before every disk read and reset call.
- Set AH immediately before the interrupt; do not assume a previous procedure left it unchanged.
- Use a destination segment:offset pair whose physical-address calculation equals 0x8000.
- Confirm that the DAP itself is not overwritten by the incoming sectors.

Bounded retry state machine

```
attempt_count = 3
read_again:
    issue extended read
    if carry is clear: success
    save the BIOS status byte
    reset the disk controller
    decrement attempt_count
    if attempts remain: read_again
    print final status and halt
```

This pseudocode is intentionally incomplete. Decide where the counter and final status are stored so that the reset interrupt does not destroy them. “Three total attempts” means the first attempt plus at most two retries.

Interpreting BIOS success and failure

For INT 0x13 services, the carry flag is the primary success/failure indication. AH commonly contains a status code after failure. Capture the status before calling another BIOS disk service, because the later service may replace AH.

Stage 2 magic validation

After a successful read, compare the four bytes at physical address 0x8000 against S, T, G, and 2. Perform all four comparisons or compare an equivalent 32-bit value only if the current processor mode and alignment assumptions are carefully understood. On any mismatch, print a validation error and halt instead of jumping into the buffer.

Entry-location reasoning

The first four bytes of stage2.bin are data, not executable instructions. Therefore, the executable entry begins immediately after the header. Document the exact runtime address and transfer to that location only after validation succeeds.

Common failure: Jumping to 0x8000 causes the processor to decode the ASCII bytes STG2 as instructions. The resulting behaviour is unpredictable and usually fails before Stage 2 can print anything.

Task 3.2 - Stage 2 loading the kernel

Stage 2 must reinitialize its own DS, ES, SS, SP, and direction flag. Do not rely on values inherited from Stage 1. The header consumes the first four bytes, so place the first executable Stage 2 instruction at the documented address after the header.

Kernel DAP differences

Reuse the same DAP concept but update the sector count, destination, and starting LBA. The kernel destination 0x10000 cannot be represented as a 16-bit offset with segment zero, so choose a valid segment:offset representation.

DAP item	Stage 1 -> Stage 2	Stage 2 -> Kernel
Sector count	4	32
Starting LBA	1	5
Physical destination	0x8000	0x10000
Expected magic	STG2	KRNL

Retry-code reuse

Conceptually reuse the three-attempt policy, but do not copy code without checking the new DAP address, saved-drive variable, error messages, and register assumptions. Stage 2 has more space, so clarity is preferable to extreme byte optimization.

Kernel magic validation

Check the first four bytes at 0x10000 for KRNL. The executable kernel entry is 0x10004. Validation proves that the beginning of the expected kernel image is present at the intended address; it does not verify every kernel byte, which is acceptable for this laboratory checkpoint.

A20 address-line study note

The A20 line permits addresses above the first megabyte to remain distinct instead of wrapping around. Although the chosen kernel and stack addresses are below 1 MiB, enabling A20 is a standard prerequisite before moving toward a protected-mode environment and future higher-memory use. Implement one recognized A20-enabling method and document which method you chose.

A20 implementation hint

The fast A20 method commonly uses system control port 0x92. The safe conceptual sequence is to read the current port value, set the A20-enable bit, avoid unintentionally asserting a reset bit, and write the modified value back. Do not write an unexplained constant to the port.

Global Descriptor Table concepts

The minimum GDT contains three descriptors: a mandatory null descriptor, a 32-bit code descriptor, and a 32-bit data descriptor. A flat model uses base zero and a limit covering the 32-bit address space, with appropriate access and granularity flags.

GDT entry	Purpose	Important properties
Null descriptor	Selector zero must be invalid	All bytes zero
Code descriptor	Execute protected-mode code	Present, executable, readable, 32-bit
Data descriptor	Data segments and stack	Present, writable, 32-bit

Selector reasoning

A selector identifies a descriptor-table entry. Because each descriptor occupies eight bytes, the code and data selector values are related to their descriptor indices. Derive the selectors from the table layout rather than memorizing unexplained constants.

GDTR structure

LGDT reads a pseudo-descriptor containing the GDT size minus one and the linear address of the first GDT byte. The “minus one” is easy to miss. Verify the generated bytes in a listing if protected-mode entry immediately resets the virtual machine.

Protected-mode transition sequence

1. Disable maskable interrupts while the system has no protected-mode interrupt table.
2. Ensure A20 is enabled.
3. Load the GDT using LGDT.
4. Read CR0, set only the PE bit, and write CR0 back.
5. Perform a far jump using the code selector and a protected-mode label.
6. Assemble subsequent instructions as 32-bit code.
7. Load the data selector into DS, ES, FS, GS, and SS.
8. Initialize ESP to the required protected-mode stack address.

9. Transfer to 0x10004, not to the KRNL header at 0x10000.

Why the far jump is essential

Setting CR0.PE changes the processor mode, but the code-segment cache still reflects the previous execution context. The far jump loads CS with the new code selector and begins execution at a protected-mode label, also discarding prefetched real-mode instructions.

32-bit entry checkpoint hints

```
[bits 32]
protected_mode_entry:
    load the data selector into all data/stack segment registers
    set ESP to the planned stack top
    TODO: call or jump to physical address 0x10004
```

This fragment deliberately omits the actual register operations and selector constants. Compute them from your GDT design.

Debugging a reset or blank screen during mode switch

- Confirm that Stage 2 printed its final real-mode checkpoint message before switching modes.
- Verify the GDT base address and limit in the assembler listing or debugger.
- Check that the far-jump selector points to the code descriptor, not the data descriptor.
- Check the descriptor present bit, executable bit, default operand size, and granularity.
- Confirm that the protected-mode label is assembled under [bits 32].
- Load SS before using the new stack and ensure ESP points to mapped RAM.
- Confirm that the kernel binary begins with KRNL and that executable code starts at 0x10004.

Recommended Stage 3 progress messages

During development, print short messages after Stage 1 begins, Stage 2 validates, Stage 2 begins, and the kernel validates. These messages localize the failure. Do not attempt BIOS output after the far jump; the kernel checkpoint should prove protected-mode execution using VGA memory.

Task 4: Build a 32-bit Protected-Mode Mini Kernel

Goal of the study material: First prove protected-mode execution with a small assembly checkpoint, then construct a freestanding C environment with a VGA terminal, PS/2 keyboard input, bounded command buffer, parser, and required shell commands.

Task 4.1 - Assembly checkpoint kernel

The checkpoint isolates the loader and mode-switching logic from the later C toolchain. It begins at origin 0x10000, contains KRNL in the first four bytes, and places the first executable instruction at 0x10004. It writes the required success message directly to VGA text memory.

Understanding a VGA text cell

In the standard 80x25 color text mode, physical address 0xB8000 refers to the first screen cell. Each cell occupies two bytes: the low byte is the character, and the high byte is the color attribute.

```
16-bit VGA cell = (attribute << 8) | character
```

Attribute bits	Meaning
Low 4 bits	Foreground color 0-15
Next 3 bits	Background color 0-7
High bit	Blink or background-intensity behaviour, depending on mode

Checkpoint printing-loop hints

- Put 0xB8000 in a destination pointer register.
- Put the address of a zero-terminated message in a source pointer register.
- Clear the direction flag before using a string-load instruction.
- Keep the chosen attribute in the high byte of a 16-bit register and load each character into its low byte.
- Write the 16-bit character/attribute pair and advance the VGA pointer by two bytes.
- Stop at the zero terminator and enter a permanent halt loop.

Interpreting the supplied checkpoint pattern

The four-byte KRNL header occupies 0x10000-0x10003. A jump instruction at 0x10004 may redirect to a later `kernel_entry` label, but Stage 2 should still transfer to 0x10004 because that is the documented executable start. The jump allows data layout and executable layout to remain explicit.

Checkpoint testing strategy

1. Test that Stage 1 and Stage 2 messages appear.
2. Verify that “kernel loaded and validated” appears before protected mode.
3. Verify that the protected-mode message appears using direct VGA access.
4. Temporarily change the color attribute to confirm that the VGA write, rather than BIOS output, produced the message.
5. Only after this succeeds, replace the checkpoint kernel with the assembly-plus-C kernel.

Task 4.2 - kernel_entry assembly responsibilities

A freestanding C function cannot assume that a normal runtime has initialized global memory. The entry assembly must provide the minimal environment expected by C: a correct stack from Stage 2, a cleared direction flag, a zeroed `.bss` region, and a call to `kernel_main`.

KRNL header and entry ordering

Place the KRNL bytes in a dedicated first section and place `kernel_entry` in a dedicated entry-text section immediately afterward. The linker script, not source-file order alone, should guarantee that the header is at 0x10000 and `kernel_entry` is at 0x10004.

Clearing .bss

The `.bss` section represents zero-initialized global and static objects, but those zeros are normally supplied by an operating-system loader. Here, define linker symbols for the beginning and end of `.bss`, calculate the byte count, and clear that range before calling C. Use a byte-wise or word-wise string operation after clearing the direction flag.

```
obtain __bss_start and __bss_end from linker symbols
destination = __bss_start
count = __bss_end - __bss_start
value = 0
TODO: clear exactly count bytes
```

The code above is algorithmic guidance only. Be careful if using a word-sized `clear`: handle alignment and any remainder, or guarantee the section alignment and size in the linker script.

C calling convention hint

With the common 32-bit C calling convention and a no-argument function, a direct `CALL` to `kernel_main` is sufficient once `ESP` is valid. If `kernel_main` returns unexpectedly, execution must enter a safe halt loop rather than falling into unrelated bytes.

Task 4.3 - Freestanding C design

Compile without hosted libraries and write the small facilities you need. Use fixed-width integer typedefs or carefully chosen built-in C types, because the usual standard headers may not be available under the assignment restrictions.

Suggested kernel modules within kernel.c

Logical module	Responsibilities
Low-level I/O	inb/outb helpers using inline assembly
VGA terminal	screen buffer, cursor, color, scrolling
String helpers	length, comparison, prefix/argument handling
Number helpers	decimal/hex parsing and formatting
Keyboard	polling, scan-code translation, Shift state
Shell	bounded line input, parsing, command dispatch

The assignment permits one kernel.c file, but structuring the code into small static functions makes it easier to test and explain.

Freestanding restrictions

- Do not include `stdio.h` or call `printf`; there is no terminal library beneath the kernel.
- Do not use `malloc`; no heap allocator exists unless you implement one, which is outside this task.
- Do not make operating-system system calls; this program is itself the operating-system environment.
- Avoid compiler-generated dependencies such as stack-protector checks, position-independent thunks, or implicit library routines by using the required compiler options.

Task 4.4 - VGA terminal architecture

Maintain terminal state in a few global variables: current row, current column, and current foreground color. Represent the VGA buffer as a pointer to volatile 16-bit cells. Volatile is important because writes have hardware-visible effects and must not be optimized away as ordinary unused memory stores.

Screen indexing

```
cell_index = row * 80 + column
vga_buffer[cell_index] = (color_attribute << 8) | character
```

Use unsigned types large enough to hold indices from 0 through 1999. Validate row and column changes after every character operation.

Clearing the screen

Fill every one of the 80 x 25 cells with a space character using the current or default color. Then reset row and column to zero and update the visible hardware cursor.

Character-output decision tree

```
if character is carriage return:
    column = 0
else if character is newline:
    column = 0; row++
else if character is backspace:
    erase the previous editable cell if one exists
else:
    write cell; column++
if column reached 80:
    column = 0; row++
if row reached 25:
    scroll upward
update hardware cursor
```

You must decide whether carriage return and newline are handled separately or whether your string-output routines always use a combined newline convention. Keep the behaviour consistent.

Scrolling algorithm

1. Copy row 1 into row 0, row 2 into row 1, and so on through the final visible row.
2. Fill the new final row with blank cells using the current color.
3. Set the logical row to the last row and preserve a valid column.
4. Update the hardware cursor after the copy is complete.

Overlap note: Because the copy moves data toward lower addresses, iterating from the top toward the bottom is safe. Reversing the direction can overwrite source cells before they are copied.

Backspace design

Backspace should move to the previous logical cell, write a blank there, and place the cursor at that position. Decide whether the shell will prevent backspacing into the prompt; a simple approach is to let the input loop track the command-buffer length and only call terminal backspace when that length is greater than zero.

Hardware-cursor movement

The VGA cursor position is a 16-bit cell index. Send the low and high bytes through the VGA CRT controller index/data ports. Encapsulate this in one function so the rest of the terminal only updates row and column.

```
position = row * 80 + column
select low-cursor register; output low byte
select high-cursor register; output high byte
```

Look up or use the controller register numbers provided in your course notes. The complete output sequence is intentionally omitted here.

Foreground color

Validate the requested value in the inclusive range 0-15. Keep the background fixed unless you deliberately extend the design. Updating the color variable affects future characters; it does not automatically recolor characters already displayed.

Task 4.5 - PS/2 keyboard polling

The polling loop first reads the controller status port. Only when the output-buffer-full bit indicates data availability should it read the scan code from the data port. Reading the data port unconditionally can return stale or unrelated values.

Polling algorithm

```
repeat until data is available:
    status = inb(0x64)
    test the output-buffer-full bit
scan_code = inb(0x60)
classify scan_code as press, release, or extended prefix
update Shift state or translate supported key
```

Scan-Code Set 1 study note

A make code represents a key press. For many ordinary keys, the release code is the make code with bit 7 set. Left and right Shift require state tracking: set the corresponding flag on press and clear it on release. Ignore unsupported releases and extended codes safely rather than placing arbitrary bytes in the command buffer.

Key category	Required action
Ordinary supported key press	Translate through normal or shifted map
Left/Right Shift press	Set the appropriate Shift flag
Left/Right Shift release	Clear the appropriate Shift flag
Enter press	Finish the current command line
Backspace press	Remove one buffered character if possible
Unsupported release/extended code	Ignore safely

Translation-table strategy

Create two lookup tables indexed by scan code: one for unshifted characters and one for shifted characters. Keep unsupported entries zero. This is clearer and less error-prone than a long chain of conditionals, but a switch statement is also acceptable if carefully bounded.

Bounded command buffer

Let the array capacity be N. Store at most N-1 typed characters so that the Enter handler can always append a final zero terminator. When the buffer is full, ignore additional printable keys or provide a short visual indication; never write past the array.

```
if printable character and length < capacity - 1:
    buffer[length] = character
    length++
    echo character
else:
    do not modify memory beyond the buffer
```

Task 4.6 - Command parsing and dispatch

When Enter is pressed, append a zero byte, print a newline, and parse the line. Skip optional leading spaces, identify the first word as the command, and preserve the remaining text as the argument region. After execution, reset the buffer length and print a new prompt.

Parser decomposition

1. Trim or skip leading spaces.
2. If the resulting string is empty, print another prompt without error.
3. Locate the end of the first word and temporarily terminate it with zero.
4. Skip spaces after the command to find the argument text.
5. Compare the command word against supported names.
6. Dispatch to exactly one command handler.
7. For an unknown command, print an informative message.

String-helper hints

Implement only the operations you need: equality comparison, prefix/argument separation, and perhaps length. Each helper must stop at a zero terminator and must not read beyond the command buffer.

Command-by-command implementation hints

Command	Detailed hint without full solution
help	Print one static line per supported command. Reuse terminal string output.
clear	Call the screen-clear routine and do not print the prompt until the screen state is reset.
echo <text>	Print the argument pointer exactly as parsed; preserve internal spaces in the remaining text.
info	Print fixed strings describing StudentOS name/version, 32-bit protected mode, and VGA/PS2 console.
regs	Read CR0 and ESP with small inline-assembly helpers; print each as hexadecimal.
color <0-15>	Parse a decimal integer, reject missing/trailing invalid text and values outside 0-15, then update the color.
peek <address>	Parse decimal or 0x-prefixed hexadecimal, cast to a volatile byte pointer, read one byte, and print it in hex.
reboot	Wait until the 8042 input buffer is ready, then send the reset command through the controller.
halt	Disable interrupts and execute a permanent halt loop.

Reading CR0 and ESP

Use inline assembly with an output operand rather than hard-coding values. CR0 confirms that protected mode is active because its PE bit is set. ESP reveals the current stack pointer after calls have placed return addresses and local data on the stack, so it may be slightly below the initial 0x90000 value.

Number formatting

Implement a reusable hexadecimal-output routine for 32-bit values. Print a fixed 0x prefix, then either print all eight hexadecimal digits or suppress leading zeros while ensuring zero itself still prints as 0. For the assignment's "regs" and "peek" output, consistency is more important than compactness.

Parsing color

Reject an empty argument. Accumulate decimal digits using $\text{value} = \text{value} * 10 + \text{digit}$, but stop and reject if a non-digit appears. Check the range after parsing. Because only 0-15 is valid, overflow is unlikely, yet the parser should still have an explicit bound or use a sufficiently wide unsigned type.

Parsing peek addresses

Detect a 0x or 0X prefix and select base 16; otherwise use base 10. Validate every digit against the selected base. Do not dereference until parsing has succeeded. QEMU generally maps low memory used by the lab, but arbitrary addresses may still be unsafe; the assignment expects a simple direct read, so document that invalid hardware/unmapped addresses can cause a fault.

Reboot command reasoning

The 8042 controller has an input-buffer status bit. Wait until it can accept a command, then issue the processor-reset command. Place this low-level sequence in a small helper and prevent the compiler from reordering port operations by using appropriate volatile inline assembly.

Unknown and empty commands

An empty command should simply produce a new prompt. An unsupported non-empty command should print the command name or a clear “unknown command” message and suggest help. Avoid using an uninitialized argument pointer when the line contains only one word.

Task 4.7 - Linker script and build pipeline

The linker script must enforce the exact in-memory layout expected by Stage 2. The first output bytes must be KRNL, the entry code must immediately follow, then normal text and data sections may continue. The .bss section occupies memory but normally contributes no initialized bytes to the binary.

Conceptual linker layout

```
set location counter to 0x10000
place the dedicated KRNL header section
assert or verify that exactly four bytes were emitted
place dedicated entry text section
place remaining .text
place .rodata and .data
define __bss_start
place .bss and COMMON objects
define __bss_end
discard unneeded hosted-runtime sections
```

This is not a complete linker script. You must supply correct linker-script syntax, section patterns, alignment choices, and discard rules.

Why entry assembly must come first

The compiler and linker may reorder ordinary .text sections. Giving the entry code a dedicated section and explicitly placing that section first prevents kernel_main or helper functions from occupying 0x10004.

Compiler options and their purposes

Option	Reason
-m32	Generate 32-bit x86 code
-ffreestanding	Do not assume a hosted C environment
-fno-pie	Avoid position-independent executable assumptions
-fno-stack-protector	Avoid references to unavailable stack-check runtime
-fno-builtin	Prevent implicit replacement with library routines
-nostdlib	Do not link host startup files or standard libraries

Build pipeline

```
assemble kernel_entry.asm -> 32-bit ELF object
compile kernel.c -> 32-bit freestanding ELF object
link both objects with linker.ld -> kernel ELF image
inspect ELF sections and symbols
convert loadable content -> raw kernel.bin
pad kernel.bin to exactly 16384 bytes
concatenate Stage 1 + Stage 2 + kernel -> raw disk image
```

Inspection before converting to a raw binary

Use tools such as readelf, objdump, nm, or a linker map to verify the header address, entry-symbol address, section order, and .bss symbols. The final raw binary should begin with the ASCII bytes KRNL, and its fifth byte should be the first byte of executable entry code.

Padding the kernel

Check the unpadding kernel size before padding. If it exceeds 16384 bytes, padding cannot fix the problem; reduce the kernel or change the assignment design only with instructor approval. If it is smaller, append zeros to reach exactly 16384 bytes and verify the final size.

Constructing the disk image

Concatenate components in disk order only after each component has the exact required size. Verify the final disk-image size and inspect the first few bytes at the LBA boundaries. The Stage 2 magic should appear at byte offset 512, and the kernel magic should appear at byte offset 5 x 512.

Boundary verification: Use xxd with an explicit seek/offset near each LBA boundary. This catches a wrong component size before QEMU debugging becomes necessary.

Integration and Debugging Strategy

Checkpoint-by-checkpoint integration

Checkpoint	Expected evidence	Likely fault area if missing
Stage 1 starts	First Stage 1 message	BIOS signature or initial real-mode code
Stage 2 validated	Stage 1 success message	DAP, read retries, STG2 header
Stage 2 starts	Stage 2 message	Wrong entry address after STG2
Kernel validated	Stage 2 success message	Kernel DAP, LBA, destination, KRNL
Protected-mode checkpoint	VGA success message	A20/GDT/CR0/far jump/segments
C heading	StudentOS heading	Linker order, .bss clear, C call
Interactive prompt	Keyboard input and commands	PS/2 polling, buffer, parser

QEMU debugging habits

- Run from a terminal so firmware messages and exit behaviour are visible.
- Use the same explicit raw-disk configuration for every test.
- Keep screenshots only after the corresponding binary has been rebuilt and verified.
- When a change breaks the system, return to the last checkpoint binary rather than modifying several layers at once.
- Use QEMU monitor or GDB support only after basic print checkpoints and binary inspection have narrowed the problem.

Common symptoms and likely causes

Symptom	Likely causes to investigate
BIOS says no bootable device	Wrong final signature or wrong drive configuration
Random characters in Stage 1	Incorrect DS, missing terminator, wrong BIOS function
Disk read always fails	Malformed DAP, wrong DS:SI, lost DL, unsupported drive configuration
Stage 2 magic mismatch	Wrong LBA, count, destination, padding, or image concatenation
Stage 2 message missing after validation	Jumped to header rather than entry, wrong runtime address
Reset immediately after PE bit	Invalid GDT, selector, far jump, or protected-mode encoding
Kernel checkpoint text garbled	Wrong VGA address, pointer increment, or AX character/attribute order
C globals contain garbage	.bss not cleared or linker symbols wrong

Symptom	Likely causes to investigate
Keyboard repeats/prints releases	Release codes not ignored or Shift state mishandled
Typing long command corrupts shell	Missing capacity-1 bound or terminator space

Binary-inspection checklist

- Stage 1: exactly 512 bytes and ending in 55 AA.
- Stage 2: exactly 2048 bytes and beginning in STG2.
- Kernel: exactly 16384 bytes and beginning in KRNL.
- Raw disk image: components appear at byte offsets 0, 512, and 2560.
- kernel_entry symbol resolves to 0x10004 in the linked ELF.
- .bss start and end symbols are ordered and lie after initialized sections.

Report-Writing Guidance

The report should demonstrate reasoning, not only screenshots. For each task, state the design, show the relevant command/output, and explain why the observed result follows from the processor or BIOS behaviour.

Suggested report structure

1. Objective and environment: tools and QEMU command used.
2. Task 1: source overview, binary-size evidence, region table, instruction-byte table, prediction versus observation, and one-byte patch explanation.
3. Task 2: procedure contracts, register-preservation decisions, dynamic boot-drive output, and test cases for hexadecimal conversion.
4. Task 3: disk/memory layout, DAP fields, retry policy, magic validation, GDT design, and protected-mode transition evidence.
5. Task 4: checkpoint kernel, C entry responsibilities, terminal design, keyboard state handling, parser design, command tests, and linker/build verification.
6. Problems encountered and how binary evidence or checkpoints were used to solve them.
7. Conclusion describing the full path from BIOS to the interactive mini kernel.

Evidence tables students should prepare themselves

Table	Minimum content
Task 1 binary regions	Start offset, end offset, purpose for code/message/padding/signature
Task 1 instruction mapping	At least five file offsets, bytes, instructions, meanings
Procedure contracts	Input, output, preserved/clobbered registers
Disk layout	LBA ranges, sector counts, byte sizes, destinations
GDT summary	Descriptor index, selector, access purpose
Command tests	Input, expected result, observed result

What not to submit as explanation

- “It works” without explaining the required address, byte layout, or processor state.
- A copied hexadecimal dump without marked regions and offsets.
- A screenshot without the command that produced it.
- Hard-coded boot-drive output presented as dynamic output.